
Exercise: Creating a User Profile Component in React

Objective

- Understand and implement the **Mounting Phase** methods in the React component lifecycle.
 - Build a simple interactive component that displays a user's profile information.
-

Setup (20 Minutes)

1. Initialize the React App:

Open a terminal and run:

```
npx create-react-app user-profile-app  
cd user-profile-app  
npm start
```

-
- The development server should start, and the default React page will load in your browser.

2. Set Up Files:

- In the `src` folder, create a new file `UserProfile.js`.
-

Part 1: Implementing the Mounting Phase (40 Minutes)

1. Create the `UserProfile` Component:

Open `UserProfile.js` and add the following code:

```
import React from "react";  
  
class UserProfile extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { name: "Loading...", age: null };  
  }  
}
```

```

    }

    static getDerivedStateFromProps(props, state) {
      if (props.name && props.name !== state.name) {
        return { name: props.name, age: props.age };
      }
      return null;
    }

    componentDidMount() {
      console.log("UserProfile component has mounted!");
      setTimeout(() => {
        this.setState({ name: "John Doe", age: 25 });
      }, 2000);
    }

    render() {
      return (
        <div>
          <h1>User Profile</h1>
          <p>Name: {this.state.name}</p>
          <p>Age: {this.state.age}</p>
        </div>
      );
    }
  }
}

export default UserProfile;

```

○

2. Integrate the Component:

In `App.js`, replace the default code with:

```

import React from "react";
import UserProfile from "../UserProfile";

```

```

function App() {
  return (
    <div className="App">
      <UserProfile />
    </div>
  );
}

```

export default App;

○

3. Run and Observe:

- Save the files and see the output in the browser.
 - Initially, the user profile should show "Loading...". After 2 seconds, it updates to "John Doe" and age 25.
-

Part 2: Enhancing the Component (30 Minutes)

1. Add Props:

Modify `App.js` to pass props to `UserProfile`:

```
function App() {  
  return (  
    <div className="App">  
      <UserProfile name="Jane Smith" age={30} />  
    </div>  
  );  
}
```

○

- Observe how the initial values are updated using `getDerivedStateFromProps`.

2. Customize with Styling:

Add some CSS to `App.css` to style the profile:

```
.App {  
  text-align: center;  
  font-family: Arial, sans-serif;  
}
```

```
div {  
  border: 1px solid #ccc;  
  padding: 20px;  
  margin: 20px;  
  border-radius: 5px;
```

```
box-shadow: 2px 2px 10px rgba(0, 0, 0, 0.1);  
display: inline-block;  
}
```

○

Part 3: Reflection and Questions (30 Minutes)

1. Group Discussion:

- How does the `constructor` help initialize the component state?
 - What is the role of `getDerivedStateFromProps` in updating the state?
 - Why do we use `componentDidMount` for side effects like data fetching?
-